```
import os, re, random, urllib.request

PATH = "sms_spam.tsv"

if not os.path.exists(PATH):
    print("File nahi mili, download ho rahi hai...")
    URL = "https://raw.githubusercontent.com/justmarkham/pycon-2016-tutorial/master/data/sms.tsv"
    urllib.request.urlretrieve(URL, PATH)

docs, labels = [], []
for line in open(PATH, encoding="utf-8"):
    line = line.rstrip("\n")
    if not line:
        continue
    label, text = line.split("\t", 1)
    labels.append(label)
    docs.append(text)

print(len(docs), "messages loaded")
print("spam:", labels.count("spam"), " ham:", labels.count("ham"))
print("\nexample spam ->", docs[labels.index("spam")][:90], "...")
```

```
File nahi mili, download ho rahi hai...
5574 messages loaded
spam: 747  ham: 4827

example spam -> Free entry in 2 a wkly comp to win FA Cup final tkts 21st May 2005. Text FA to 87121 to re ...
```

```
random.seed(42)
order = list(range(len(docs)))
random.shuffle(order)
docs   = [docs[i]   for i in order]
labels = [labels[i] for i in order]

cut = int(0.8 * len(docs))
train_docs,  test_docs   = docs[:cut],   docs[cut:]
train_labels, test_labels = labels[:cut], labels[cut:]
print("train:", len(train_docs), " | test:", len(test_docs))
```

```
train: 4459  | test: 1115
```

```
class MultinomialNaiveBayes:
    def __init__(self, alpha=1):
        self.alpha = alpha                # add-1 smoothing => alpha = 1

    def _tokenize(self, text):
        return re.findall(r"[a-z0-9]+", text.lower())  # given: real tokenizer

    def fit(self, docs, labels):
        self.classes = sorted(set(labels))
        self.vocab = set()
        self.doc_count = {c: 0 for c in self.classes}       # docs per class (for prior)
        self.word_counts = {c: {} for c in self.classes}    # word -> count, per class
```

```
        self.total_words = {c: 0 for c in self.classes}    # total tokens, per class
        n_docs = len(docs)

        for text, c in zip(docs, labels):
            tokens = self._tokenize(text)
            self.doc_count[c] += 1
            for w in tokens:
                # TODO 1: add w to self.vocab
                self.vocab.add(w)

                # TODO 2: add 1 to self.word_counts[c][w] (use .get(w, 0), it may be new)
                self.word_counts[c][w] = self.word_counts[c].get(w, 0) + 1

                # TODO 3: add 1 to self.total_words[c]
                self.total_words[c] += 1

        self.vocab_size = len(self.vocab)

        # TODO 4: prior of each class = docs in that class / total docs
        self.priors = {}
        for c in self.classes:
            self.priors[c] = self.doc_count[c] / n_docs

        return self

    def _likelihood(self, word, c):
        """add-1 smoothed P(word | c)."""
        count = self.word_counts[c].get(word, 0)

        # TODO 5: return (count + alpha) / (total_words[c] + alpha * vocab_size)
        return (count + self.alpha) / (self.total_words[c] + self.alpha * self.vocab_size)

    def predict_one(self, text):
        tokens = self._tokenize(text)
        best_class, best_score = None, -1.0

        for c in self.classes:
            score = self.priors[c]  # start from the prior
            for w in tokens:
                if w in self.vocab:  # drop unknown words
                    # TODO 6: multiply score by the likelihood of w given c
                    score *= self._likelihood(w, c)

            # TODO 7: if score beats best_score, remember this class
            if score > best_score:
                best_score = score
                best_class = c

        return best_class

    def predict(self, docs):
        return [self.predict_one(t) for t in docs]
```

```
def accuracy(model, docs, labels):
    preds = model.predict(docs)
```

```
    return sum(p == y for p, y in zip(preds, labels)) / len(labels)
```

```
model = MultinomialNaiveBayes(alpha=1).fit(train_docs, train_labels)
print("vocabulary size:", model.vocab_size)
print("priors:", {c: round(p, 3) for c, p in model.priors.items()})

assert model.vocab_size > 5000, "real vocab should be in the thousands"
assert abs(sum(model.priors.values()) - 1) < 1e-9, "priors must sum to 1"
assert model.priors["ham"] > model.priors["spam"], "ham should be the majority class"
print("✅ fit looks correct")
```

```
vocabulary size: 7725
priors: {'ham': 0.868, 'spam': 0.132}
✅ fit looks correct
```

```
acc = accuracy(model, test_docs, test_labels)
print(f"test accuracy: {acc:.4f}")
assert acc > 0.95, "Naive Bayes should score well above 95% on this dataset"
print("✅ predict works — accuracy above 95%")
```

```
test accuracy: 0.9830
✅ predict works — accuracy above 95%
```

```
for msg in ["congratulations you won a free prize call now to claim",
            "hey are we still on for dinner tonight",
            "URGENT your account has won 1000 cash reply now"]:
    print(f"{model.predict_one(msg):>4}  <-  {msg}")

# which words most strongly signal spam?
ratio = {w: model._likelihood(w, "spam") / model._likelihood(w, "ham") for w in model.vocab}
print("\nmost spam-indicative words:", sorted(ratio, key=ratio.get, reverse=True)[:8])
```

```
spam  <-  congratulations you won a free prize call now to claim
 ham  <-  hey are we still on for dinner tonight
spam  <-  URGENT your account has won 1000 cash reply now

most spam-indicative words: ['claim', 'prize', '150p', 'tone', '18', 'guaranteed', 'cs', '1000']
```

```
long_email = " ".join([t for t, y in zip(train_docs, train_labels) if y == "spam"][:6])
print("length in tokens:", len(model._tokenize(long_email)))

score = model.priors["spam"]
for w in model._tokenize(long_email):
    if w in model.vocab:
        score *= model._likelihood(w, "spam")
print("plain product score:", score, "  <- 0.0 means it underflowed to nothing")
```

```
length in tokens: 165
plain product score: 0.0   <- 0.0 means it underflowed to nothing
```

```
import math

class LogNaiveBayes(MultinomialNaiveBayes):
    def predict_one(self, text):
```

```
        tokens = self._tokenize(text)
        best_class, best_score = None, float("-inf")
        for c in self.classes:
            # TODO A: start score at log(prior of c)
            score = math.log(self.priors[c])

            for w in tokens:
                if w in self.vocab:
                    # TODO B: ADD log(likelihood of w given c) to score
                    score += math.log(self._likelihood(w, c))

            # TODO C: keep the highest-scoring class
            if score > best_score:
                best_score = score
                best_class = c

        return best_class
```

```
log_model = LogNaiveBayes(alpha=1).fit(train_docs, train_labels)
log_acc = accuracy(log_model, test_docs, test_labels)
print(f"log-space accuracy: {log_acc:.4f}")
print("long email classified as:", log_model.predict_one(long_email))

assert log_acc > 0.95, "log version should be just as accurate"
assert log_model.predict_one(long_email) == "spam", "log version must handle the long email"
print("✅ log version works and survives the document that underflowed above")
```

```
log-space accuracy: 0.9830
long email classified as: spam
✅ log version works and survives the document that underflowed above
```

```
# Your answers (1-4):
# 1. Giving unknown words probability 0 would zero out the total probability product for ALL classes.
#    Dropping them means unknown words carry neutral information and don't affect classification.

# 2. Add-1 (Laplace) smoothing prevents the zero-frequency problem for known words that never appeared
#    in a specific class during training. If alpha = 0, any unseen word-class pair gets P(w|c) = 0,
#    which destroys the class probability regardless of other strong evidence.

# 3. The prior tips the decision heavily towards 'ham'. Because P(ham) = 0.87 vs P(spam) = 0.13,
#    a neutral message without strong spam indicators will naturally default to 'ham' since its prior
#    probability is much higher.

# 4. Long documents involve multiplying hundreds of small fractional probabilities (0 < p < 1).
#    Without log transformation, this product floating-point underflows to 0.0, causing ties and incorrect predictions.
```

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB

vec = CountVectorizer(lowercase=True, token_pattern=r"[a-z0-9]+")  # match our tokenizer

# TODO 1: fit_transform the TRAINING docs into a count matrix
X_train = vec.fit_transform(train_docs)
```

```
# TODO 2: create MultinomialNB(alpha=1) and fit it on X_train, train_labels
clf = MultinomialNB(alpha=1.0)
clf.fit(X_train, train_labels)

# TODO 3: transform the TEST docs and predict
X_test = vec.transform(test_docs)
sk_preds = clf.predict(X_test)

print("first 8 scikit-learn predictions:", list(sk_preds)[:8])
```

```
first 8 scikit-learn predictions: [np.str_('ham'), np.str_('ham'), np.str_('ham'), np.str_('spam'), np.str_('spam'), np.str_('spam'), np.str_('ham'), np.str_('ham')]
```

```
sk_acc = sum(str(p) == y for p, y in zip(sk_preds, test_labels)) / len(test_labels)
my_preds = log_model.predict(test_docs)
agree = sum(str(a) == b for a, b in zip(sk_preds, my_preds)) / len(sk_preds)

print(f"scikit-learn accuracy      : {sk_acc:.4f}")
print(f"agreement with your model  : {agree:.4f}")
assert sk_acc > 0.95
assert agree > 0.98, "your log model should essentially match scikit-learn"
print("\n✅ Your from-scratch Naive Bayes matches scikit-learn!")
```

```
scikit-learn accuracy      : 0.9830
agreement with your model  : 1.0000

✅ Your from-scratch Naive Bayes matches scikit-learn!
```